Poker Theory & Bot Design: Lecture 3

Hand Selection, Position & the Lookup Bot

Poker Theory and Design of Poker Bots Reading Group

February 24, 2026

Meeting 3 Overview

1. Hand Strength
2. Tight vs. Loose Play & Range Analysis
3. Positional Advantage
4. Building the Lookup Bot
5. Bluffing & the 15% Surprise
6. Next Time: Bluffing Theory & the Road to GTO

Table of Contents

1. Hand Strength
2. Tight vs. Loose Play & Range Analysis
3. Positional Advantage
4. Building the Lookup Bot
5. Bluffing & the 15% Surprise
6. Next Time: Bluffing Theory & the Road to GTO

Let's Do Some Combinatorics

A standard deck: 13 ranks $\times$ 4 suits $=$ 52 cards. Each player receives 2 private ("hole") cards, unordered.

$$|\Omega| = \binom{52}{2} = \mathbf{1326} \quad \text{starting hands}$$

Suit symmetry (preflop): No community cards have been dealt, so suits are interchangeable. K♠A♡ $\equiv$ K♡A♢ strategically. This collapses to **169** canonical hand classes:

$$\begin{aligned} \text{Pairs} &: \binom{13}{1}\binom{4}{2} = 13 \times 6 = 78 \\ \text{Suited} &: \binom{13}{2} \times 4 = 312 \\ \text{Offsuit} &: \binom{13}{2} \times 12 = 936 \end{aligned}$$

Total: $78 + 312 + 936 = 1326$. Classes: $13 + 78 + 78 = \mathbf{169}$.

Hand Weights: What They Mean

Definition. The *weight* $w(h)$ of canonical hand h is the number of distinct card-pair deals in that class:

$$w(h) = \left|\{(c_1, c_2) \in \Omega : (c_1, c_2) \sim h\}\right|$$

Examples:

- $w(\mathrm{AA}) = \binom{4}{2} = 6$ (choose 2 from 4 aces: A♡A♢, A♡A♣, A♡A♠, A♢A♣, A♢A♠, A♣A♠)

Hand Weights: What They Mean

Definition. The *weight* $w(h)$ of canonical hand h is the number of distinct card-pair deals in that class:

$$w(h) = \left|\{(c_1, c_2) \in \Omega : (c_1, c_2) \sim h\}\right|$$

Examples:

- $w(\mathrm{AA}) = \binom{4}{2} = 6$ (choose 2 from 4 aces: A♡A♢, A♡A♣, A♡A♠, A♢A♣, A♢A♠, A♣A♠)
- $w(\mathrm{AKs}) = 4$ (one per suit: A♡K♡, A♢K♢, A♣K♣, A♠K♠)

Hand Weights: What They Mean

Definition. The *weight* $w(h)$ of canonical hand h is the number of distinct card-pair deals in that class:

$$w(h) = \left|\{(c_1, c_2) \in \Omega : (c_1, c_2) \sim h\}\right|$$

Examples:

- $w(\text{AA}) = \binom{4}{2} = 6$ (choose 2 from 4 aces: A♡A♢, A♡A♣, A♡A♠, A♢A♣, A♢A♠, A♣A♠)
- $w(\text{AKs}) = 4$ (one per suit: A♡K♡, A♢K♢, A♣K♣, A♠K♠)
- $w(\text{AKo}) = 4 \times 4 - 4 = 12$ (all combos minus suited)

Suited, Connected, and Drawing Hands

Suited, Connected, and Drawing Hands

Suited (s): Both hole cards share the same suit.

Suited, Connected, and Drawing Hands

Suited (s): Both hole cards share the same suit.

- Example: K♡9♡ (both hearts)

Suited, Connected, and Drawing Hands

Suited (s): Both hole cards share the same suit.

- Example: K♡9♡ (both hearts)
- Enables a *flush*

Suited, Connected, and Drawing Hands

Suited (s): Both hole cards share the same suit.

- Example: K♡9♡ (both hearts)
- Enables a *flush*

Connected (connectors): Adjacent ranks (differ by 1).

Suited, Connected, and Drawing Hands

Suited (s): Both hole cards share the same suit.

- Example: K♡9♡ (both hearts)
- Enables a *flush*

Connected (connectors): Adjacent ranks (differ by 1).

- Example: 9♢8♢

Suited, Connected, and Drawing Hands

Suited (s): Both hole cards share the same suit.

- Example: K♡9♡ (both hearts)
- Enables a *flush*

Connected (connectors): Adjacent ranks (differ by 1).

- Example: 9♢8♢
- Enables a *straight*

Hand Equity: Formal Definition

Definition (Equity). Let h_1, h_2 be two starting hands. B is drawn uniformly over all boards not conflicting with h_1, h_2.

$$\mathrm{Eq}(h_1 \mid h_2) = \Pr[\mathrm{rank}(h_1, B) > \mathrm{rank}(h_2, B)] + \frac{1}{2}\Pr[\mathrm{rank}(h_1, B) = \mathrm{rank}(h_2, B)]$$

The $\frac{1}{2}$ handles splits (ties). We can come up with a more complicated formula for more players using inclusion-exclusion principal.

Hand Equity: Formal Definition

Definition (Equity). Let h_1, h_2 be two starting hands. B is drawn uniformly over all boards not conflicting with h_1, h_2.

$$\mathrm{Eq}(h_1 \mid h_2) = \Pr[\mathrm{rank}(h_1, B) > \mathrm{rank}(h_2, B)] + \frac{1}{2}\Pr[\mathrm{rank}(h_1, B) = \mathrm{rank}(h_2, B)]$$

The $\frac{1}{2}$ handles splits (ties). We can come up with a more complicated formula for more players using inclusion-exclusion principal.

Selected equities (R code):

h_1	h_2	$\mathrm{Eq}(h_1)$
`AA`	`KK`	81.8%
`AA`	`72o`	95.7%
`AKs`	`72o`	73.4%

Hand Equity: Formal Definition

Definition (Equity). Let h_1, h_2 be two starting hands. B is drawn uniformly over all boards not conflicting with h_1, h_2.

$$\mathrm{Eq}(h_1 \mid h_2) = \Pr[\mathrm{rank}(h_1, B) > \mathrm{rank}(h_2, B)] + \tfrac{1}{2}\Pr[\mathrm{rank}(h_1, B) = \mathrm{rank}(h_2, B)]$$

The $\frac{1}{2}$ handles splits (ties). We can come up with a more complicated formula for more players using inclusion-exclusion principal.

Selected equities (R code):

h_1	h_2	$\mathrm{Eq}(h_1)$
AA	KK	81.8%
AA	72o	95.7%
AKs	72o	73.4%

Range equity. In practice we don't know opponent's exact hand. We compute equity against their *range* $\mathcal{R}$:

$$\mathrm{Eq}(h_1 \mid \mathcal{R}) = \sum_{h_2} \mathcal{R}(h_2) \cdot \mathrm{Eq}(h_1 \mid h_2)$$

Equity Is Not EV!

Equity: Who wins if we were to move to showdown immediately.
Expected Value (EV) of a call accounts for:

- **Pot odds:** you risk C to win a pot of P

Figuring out the EV is quite hard and computationally infeasible. We are trying to estimate it at varying levels of complexity. The simplest and most straightforward one being EV.

Equity Is Not EV!

Equity: Who wins if we were to move to showdown immediately.
Expected Value (EV) of a call accounts for:

- **Pot odds:** you risk C to win a pot of P
- **Fold equity:** opponent may fold before showdown; you win immediately

Figuring out the EV is quite hard and computationally infeasible. We are trying to estimate it at varying levels of complexity. The simplest and most straightforward one being EV.

Equity Is Not EV!

Equity: Who wins if we were to move to showdown immediately.
Expected Value (EV) of a call accounts for:

- **Pot odds:** you risk C to win a pot of P
- **Fold equity:** opponent may fold before showdown; you win immediately
- **Future betting streets:** more bets may come on turn and river

Figuring out the EV is quite hard and computationally infeasible. We are trying to estimate it at varying levels of complexity. The simplest and most straightforward one being EV.

Equity Is Not EV!

Equity: Who wins if we were to move to showdown immediately.
Expected Value (EV) of a call accounts for:

- **Pot odds:** you risk C to win a pot of P
- **Fold equity:** opponent may fold before showdown; you win immediately
- **Future betting streets:** more bets may come on turn and river
- **Position:** acting last gives you information your opponent doesn't have

Figuring out the EV is quite hard and computationally infeasible. We are trying to estimate it at varying levels of complexity. The simplest and most straightforward one being EV.

Pot Odds and the Call Threshold

Recall. Given Pot $= P$, call costs C:

$$\text{Pot Odds} = \frac{C}{P + C}$$

That is the minimum equity needed to break even on a call.
If we use the same idea as the one from last time, we can have a decision rule as

$$\mathrm{Eq}(h_1 \mid \hat{h}_2) \geq \frac{C}{P + C}$$

where $\hat{h}_2$ is the opponent's range

Why Are Some Hands Strong?

Why Are Some Hands Strong?

1. Raw equity dominance.

$$\mathrm{Eq}(\mathrm{AA} \mid h) \geq 0.78 \quad \forall\, h \neq \mathrm{AA}$$

2. Dominated kicker. Both players flop an Ace; whoever has the higher second card wins.

$$\mathrm{Eq}(\mathrm{A2o} \mid \mathrm{AKo}) \approx 28\%$$

Playing A2 often means paying to lose to AK.

Why Are Some Hands Strong?

1. Raw equity dominance.

$$\mathrm{Eq}(\mathrm{AA} \mid h) \geq 0.78 \quad \forall\, h \neq \mathrm{AA}$$

2. Dominated kicker. Both players flop an Ace; whoever has the higher second card wins.

$$\mathrm{Eq}(\mathrm{A2o} \mid \mathrm{AKo}) \approx 28\%$$

Playing A2 often means paying to lose to AK.

3. Drawing potential.

$$\Pr[\text{flush by river} \mid \text{suited}] \approx 5.9\%$$
$$\Pr[\text{straight} \mid \text{connectors}] \approx 4.6\%$$

High suited connectors (K♡Q♡) have both benefits.

4. Why 72o is worst.

7 and 2 cannot form any straight.
Offsuit means no flush draw.
$\mathrm{Eq}(72o \mid \text{random}) \approx 32.2\%$.

9−Handed Preflop All−In Equity (Monte Carlo)

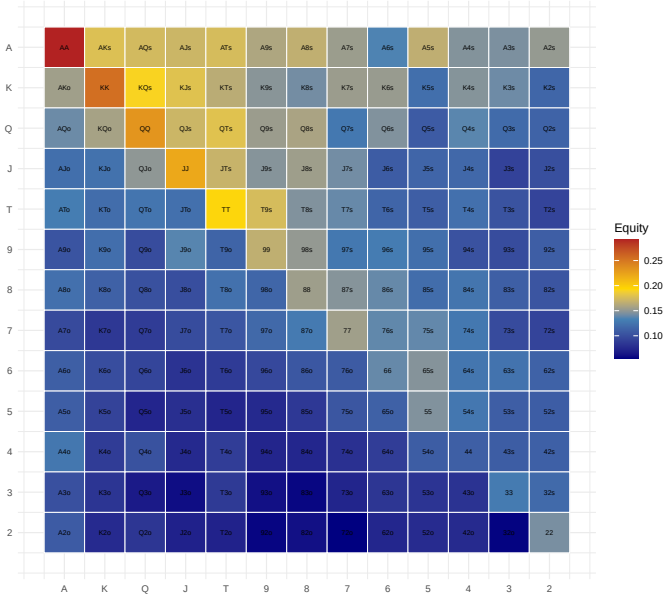


Table of Contents

1. Hand Strength
2. Tight vs. Loose Play & Range Analysis
3. Positional Advantage
4. Building the Lookup Bot
5. Bluffing & the 15% Surprise
6. Next Time: Bluffing Theory & the Road to GTO

VPIP: Voluntarily Put $ In Pot

Definition. VPIP is the fraction of hands a player *voluntarily* enters the pot with (i.e. does not fold preflop, excluding the forced blind posts):

$$\mathrm{VPIP} = \frac{\sum_{h:\,\pi(h)\neq\text{fold}} w(h)}{\sum_h w(h)} \times 100\%$$

The $w(h)$ weighting is essential: playing KKo counts 12× as often as you are dealt it compared to playing it once.

Style	VPIP	Characteristic
Nit	$< 15\%$	Only strongest premiums
Tight	15–22%	**GTO-approximate region**
Loose	23–35%	Many speculative hands
Maniac	$> 35\%$	Plays most hands

Range Analysis: Thinking in Distributions

What is a range? Since we cannot see opponent's hole cards, we maintain a probability distribution over possible hands:

$$\mathcal{R}(h) = \Pr[h_{\text{opp}} = h \mid \text{observed actions}]$$

Initialised as $\mathcal{R}(h) = w(h)/1326$ (uniform prior), then updated as actions are observed.

Bayesian update. Let A_t be the action taken at time t and $\sigma^*(A_t \mid h)$ be the GTO probability of taking action A_t with hand h:

$$\mathcal{R}(h \mid A_1, \ldots, A_t) = \frac{\mathcal{R}(h) \prod_i \sigma^*(A_i \mid h)}{\sum_{h'} \mathcal{R}(h') \prod_i \sigma^*(A_i \mid h')}$$

Example. If someone raises from UTG (first to act, 9-handed), we are expecting them to have some of the best possible hands in the game and by no means have 72o. So we can say $\mathcal{R}(72\text{o}) approx 0$.

We must train our bots to play against this distribution, not the initial $\mathcal{R}(h) = \frac{w(h)}{1326}$.

Table of Contents

1. Hand Strength
2. Tight vs. Loose Play & Range Analysis
3. Positional Advantage
4. Building the Lookup Bot
5. Bluffing & the 15% Surprise
6. Next Time: Bluffing Theory & the Road to GTO

UTG and Other Positions: What They Mean

Positions in a 9-handed game (clockwise from dealer button):

- **BTN** (Button): dealer; acts last postflop — best seat

UTG and Other Positions: What They Mean

Positions in a 9-handed game (clockwise from dealer button):

- **BTN** (Button): dealer; acts last postflop — best seat
- **SB** (Small Blind): posts half a big blind; acts second-to-last preflop, *first* postflop

UTG and Other Positions: What They Mean

Positions in a 9-handed game (clockwise from dealer button):

- **BTN** (Button): dealer; acts last postflop — best seat
- **SB** (Small Blind): posts half a big blind; acts second-to-last preflop, *first* postflop
- **BB** (Big Blind): posts 1bb; last to act preflop

UTG and Other Positions: What They Mean

Positions in a 9-handed game (clockwise from dealer button):

- **BTN** (Button): dealer; acts last postflop — best seat
- **SB** (Small Blind): posts half a big blind; acts second-to-last preflop, *first* postflop
- **BB** (Big Blind): posts 1bb; last to act preflop
- **UTG** (Under the Gun): acts *first* preflop — worst position

UTG and Other Positions: What They Mean

Positions in a 9-handed game (clockwise from dealer button):

- **BTN** (Button): dealer; acts last postflop — best seat
- **SB** (Small Blind): posts half a big blind; acts second-to-last preflop, *first* postflop
- **BB** (Big Blind): posts 1bb; last to act preflop
- **UTG** (Under the Gun): acts *first* preflop — worst position
- **MP** Middle Positions

UTG and Other Positions: What They Mean

Positions in a 9-handed game (clockwise from dealer button):

- **BTN** (Button): dealer; acts last postflop — best seat
- **SB** (Small Blind): posts half a big blind; acts second-to-last preflop, *first* postflop
- **BB** (Big Blind): posts 1bb; last to act preflop
- **UTG** (Under the Gun): acts *first* preflop — worst position
- **MP** Middle Positions
- **CO** (Cutoff): one before BTN; second-best seat

Position Edge

Definition. Player i is *in position* (IP) if i acts *after* all opponents on every postflop street. Acting first = *out of position* (OOP).

The IP player observes the OOP check/bet before responding. Let $A_{\text{OOP}} \in \{\text{check}, \text{bet}\}$ carry $H(A_{\text{OOP}}) > 0$ bits. IP's strategy can condition on this signal; OOP cannot. The IP player's expected utility weakly dominates OOP's for the same hand.

GTO solver data (6-max, 100bb deep):

Position	Meaning	EV (bb/hand)	VPIP
BTN	Button, last postflop	$+0.42$	47%
CO	Cutoff	$+0.24$	32%
UTG + 1	Hijack	$+0.11$	27%
UTG	Lojack	-0.01	22%
BB	Big Blind (forced)	-0.12	—
SB	Small Blind (forced)	-0.27	—

Position VPIP: Why We Play More Hands Late

We play *wider* ranges from later positions:

$$\text{UTG range} \approx 12\text{–}15\% \qquad \text{CO range} \approx 25\text{–}30\% \qquad \text{BTN range} \approx 40\text{–}50\%$$

Why?

- IP, you can see the OOP player check-fold and take the pot cheaply

Bot design implication. Strategy must be $\pi(h, \text{pos})$, not just $\pi(h)$. The VPIP of our LookupBot should vary by seat.

Position VPIP: Why We Play More Hands Late

We play *wider* ranges from later positions:

UTG range $\approx$ 12–15% CO range $\approx$ 25–30% BTN range $\approx$ 40–50%

Why?

- IP, you can see the OOP player check-fold and take the pot cheaply
- IP, you can control pot size on draws (call small bets, fold big ones)

Bot design implication. Strategy must be $\pi(h, \text{pos})$, not just $\pi(h)$. The VPIP of our LookupBot should vary by seat.

Position VPIP: Why We Play More Hands Late

We play *wider* ranges from later positions:

$$\text{UTG range} \approx 12\text{–}15\% \qquad \text{CO range} \approx 25\text{–}30\% \qquad \text{BTN range} \approx 40\text{–}50\%$$

Why?

- IP, you can see the OOP player check-fold and take the pot cheaply
- IP, you can control pot size on draws (call small bets, fold big ones)
- The information advantage more than compensates for weaker raw equity

Bot design implication. Strategy must be $\pi(h, \text{pos})$, not just $\pi(h)$. The VPIP of our LookupBot should vary by seat.

Table of Contents

1. Hand Strength
2. Tight vs. Loose Play & Range Analysis
3. Positional Advantage
4. Building the Lookup Bot
5. Bluffing & the 15% Surprise
6. Next Time: Bluffing Theory & the Road to GTO

Bot Architecture: Design Principles

What the Lookup Bot needs to know:

1. Current canonical hand h

Policy:

$$\pi(h, \text{pos}, \text{ctx}) \in \{\text{fold, call, raise}\}$$

Implemented as a *lookup table* no ML, no tree traversal, no bluffing.

The Lookup Bot is the simplest non-trivial agent. All future bots are benchmarked against it. Its key property: it is **deterministic** given its input, which makes it exploitable (we return to this).

Bot Architecture: Design Principles

What the Lookup Bot needs to know:

1. Current canonical hand h
2. Current position (determines which range to use)

Policy:

$$\pi(h, \text{pos}, \text{ctx}) \in \{\text{fold, call, raise}\}$$

Implemented as a *lookup table* no ML, no tree traversal, no bluffing.

The Lookup Bot is the simplest non-trivial agent. All future bots are benchmarked against it. Its key property: it is **deterministic** given its input, which makes it exploitable (we return to this).

Bot Architecture: Design Principles

What the Lookup Bot needs to know:

1. Current canonical hand h
2. Current position (determines which range to use)
3. Stack sizes and pot size (for raise sizing)

Policy:

$$\pi(h, \text{pos}, \text{ctx}) \in \{\text{fold, call, raise}\}$$

Implemented as a *lookup table* no ML, no tree traversal, no bluffing.

The Lookup Bot is the simplest non-trivial agent. All future bots are benchmarked against it. Its key property: it is **deterministic** given its input, which makes it exploitable (we return to this).

Bot Architecture: Design Principles

What the Lookup Bot needs to know:

1. Current canonical hand h
2. Current position (determines which range to use)
3. Stack sizes and pot size (for raise sizing)
4. Action history: are we facing a raise?

Policy:

$$\pi(h, \text{pos}, \text{ctx}) \in \{\text{fold, call, raise}\}$$

Implemented as a *lookup table* no ML, no tree traversal, no bluffing.

The Lookup Bot is the simplest non-trivial agent. All future bots are benchmarked against it. Its key property: it is **deterministic** given its input, which makes it exploitable (we return to this).

Lookup Bot: Ranges and Decision Engine (Python) I

```python
import random
from pypokerengine.players import BasePokerPlayer
from pypokerengine.utils.card_utils import gen_cards, estimate_hole_card_win_rate

# ============================================================
# Canonical Hand Representation (169 classes)
# ============================================================

RANK_MAP = {
    "2": 2, "3": 3, "4": 4, "5": 5, "6": 6, "7": 7,
    "8": 8, "9": 9, "T": 10, "J": 11,
    "Q": 12, "K": 13, "A": 14
}


def canonical_key(hole_card):
    # PyPokerEngine format is "SA", "DT", etc.
    r1 = RANK_MAP[hole_card[0][1]]
    r2 = RANK_MAP[hole_card[1][1]]
```

Lookup Bot: Ranges and Decision Engine (Python) II

```python
    s1 = hole_card[0][0]
    s2 = hole_card[1][0]

    hi, lo = max(r1, r2), min(r1, r2)
    suited = (s1 == s2)

    if hi == lo:
        suited = False  # pairs ignore suit

    return (hi, lo, suited)


# ============================================================
# Preflop Ranges
# ============================================================

def build_utg_range():
    pairs = {(r, r, False) for r in [14,13,12,11,10,9,8,7]}
    suited = {
        (14,13,True),(14,12,True),(14,11,True),(14,10,True),
```

Lookup Bot: Ranges and Decision Engine (Python) III

```
        (13,12,True),(13,11,True),(12,11,True),(11,10,True),(10,9,True)
    }
    offsuit = {
        (14,13,False),(14,12,False),(14,11,False)
    }
    return pairs | suited | offsuit


def build_co_range():
    R = build_utg_range()
    extra_pairs = {(6,6,False),(5,5,False)}
    extra_suited = {
        (9,8,True),(8,7,True),(7,6,True),
        (14,9,True),(14,8,True)
    }
    return R | extra_pairs | extra_suited


def build_btn_range():
    R = build_co_range()
```

Lookup Bot: Ranges and Decision Engine (Python) IV

```python
    extra_pairs = {(4,4,False),(3,3,False),(2,2,False)}
    extra_suited = {
        (6,5,True),(5,4,True),(4,3,True),
        (13,9,True),(12,9,True)
    }
    extra_offsuit = {
        (14,10,False),(13,12,False)
    }
    return R | extra_pairs | extra_suited | extra_offsuit


def build_sb_range():
    return build_btn_range()


OPENING_RANGES = {
    "UTG": build_utg_range(),
    "CO": build_co_range(),
    "BTN": build_btn_range(),
    "SB": build_sb_range()
```

Lookup Bot: Ranges and Decision Engine (Python) V

```python
}

PREMIUM = {
    (14,14,False),
    (13,13,False),
    (12,12,False),
    (14,13,True)
}


# ============================================================
# LookupBot
# ============================================================

class LookupBot(BasePokerPlayer):

    SIMULATIONS = 8000
    RAISE_THRESHOLD = 0.60

    # ------------------------------------------------------------
```

Lookup Bot: Ranges and Decision Engine (Python) VI

```python
# Position detection
# -------------------------------------------------------

def get_position(self, round_state):
    seats = round_state['seats']
    dealer_btn = round_state['dealer_btn']
    my_index = next(i for i, s in enumerate(seats)
                    if s['uuid'] == self.uuid)

    relative = (my_index - dealer_btn) % len(seats)

    if relative == 0:
        return "BTN"
    elif relative == 1:
        return "SB"
    elif relative == 2:
        return "BB"
    elif relative == 3:
        return "UTG"
    else:
```

Lookup Bot: Ranges and Decision Engine (Python) VII

```python
            return "CO"

    # -------------------------------------------------------
    # Monte Carlo Postflop Equity
    # -------------------------------------------------------

    def estimate_equity(self, hole_card, community_card, nb_players):
        return estimate_hole_card_win_rate(
            nb_simulation=self.SIMULATIONS,
            nb_player=nb_players,
            hole_card=gen_cards(hole_card),
            community_card=gen_cards(community_card)
        )

    def calculate_pot_odds(self, call_amount, pot):
        if call_amount == 0:
            return 0
        return call_amount / float(pot + call_amount)

    # -------------------------------------------------------
```

Lookup Bot: Ranges and Decision Engine (Python) VIII

```python
    # Main Decision Engine
    # -------------------------------------------------------

    def declare_action(self, valid_actions, hole_card, round_state):

        street = round_state['street']
        pot = round_state['pot']['main']['amount']
        nb_players = len(round_state['seats'])

        call_action = next(a for a in valid_actions if a['action'] == 'call')
        call_amount = call_action['amount']

        raise_action = next(a for a in valid_actions if a['action'] == 'raise')

        # ====================================================
        # PREFLOP LOOKUP ENGINE
        # ====================================================

        if street == "preflop":
```

Lookup Bot: Ranges and Decision Engine (Python) IX

```python
        position = self.get_position(round_state)
        key = canonical_key(hole_card)
        my_range = OPENING_RANGES.get(position, set())

        facing_raise = call_amount > 0

        # OPENING
        if not facing_raise:
            if key in my_range and raise_action['amount']['min'] != -1:
                return "raise", raise_action['amount']['min']
            return "fold", 0

        # FACING RAISE
        else:
            if key in PREMIUM and raise_action['amount']['min'] != -1:
                return "raise", raise_action['amount']['min'] * 3

            if key in my_range:
                return "call", call_amount
```

Lookup Bot: Ranges and Decision Engine (Python) X

```
            return "fold", 0

        # ==================================================
        # POSTFLOP MATH ENGINE
        # ==================================================

        community_card = round_state['community_card']

        equity = self.estimate_equity(hole_card, community_card, nb_players)
        pot_odds = self.calculate_pot_odds(call_amount, pot)

        if equity >= pot_odds:

            if equity >= self.RAISE_THRESHOLD and raise_action['amount']['min'] != -1:
                min_raise = raise_action['amount']['min']
                max_raise = raise_action['amount']['max']

                raise_amount = int(pot * equity)
                raise_amount = max(min_raise, min(raise_amount, max_raise))
```

Lookup Bot: Ranges and Decision Engine (Python) XI

```
                return "raise", raise_amount

            return "call", call_amount

        return "fold", 0

    # Required abstract methods
    def receive_game_start_message(self, game_info):
        pass

    def receive_round_start_message(self, round_count, hole_card, seats):
        pass

    def receive_street_start_message(self, street, round_state):
        pass

    def receive_game_update_message(self, action, round_state):
        pass

    def receive_round_result_message(self, winners, hand_info, round_state):
```

Lookup Bot: Ranges and Decision Engine (Python) XII

```
        pass
```

Variance: How Many Hands to Trust Your Win Rate?

Win rate estimator from N hands (R_i = result of hand i):

$$\hat{\mu} = \frac{1}{N}\sum_{i=1}^{N} R_i, \qquad R_i \sim (\mu, \sigma^2)$$

By CLT, a 95% confidence interval is:

$$\mu \in \hat{\mu} \pm \frac{1.96\,\sigma}{\sqrt{N}}$$

Required sample size to detect a $\mu = 5$ bb/100h win rate ($\sigma = 100$):

$$N_{\text{min}} = \left(\frac{1.96 \times 100}{5}\right)^2 \approx 1{,}537 \text{ hands}$$

But this assumes $\sigma = 100$. In reality $\sigma \approx 80$–120, so estimates are noisy over $< 10{,}000$ hands. So we must test our bots a few 10000+ times to be sure of any performance guarantees.

Table of Contents

1. Hand Strength
2. Tight vs. Loose Play & Range Analysis
3. Positional Advantage
4. Building the Lookup Bot
5. Bluffing & the 15% Surprise
6. Next Time: Bluffing Theory & the Road to GTO

A Bot That Beats Us

While LookupBot beats MathBot, it loses to a simple addition.

```
class FifteenBluffLookupBot(LookupBot):
BLUFF_FREQUENCY = 0.1515
def declare_action(self, valid_actions, hole_card, round_state) :
Get original LookupBot decision action, amount = super().declare_action(valid_actions, hole_card, round_state)
If original action is NOT fold → keep it if action != "fold": return action, amount
If original action IS fold → 15if random.random() < self.BLUFF_FREQUENCY :
call_action = next(aforainvalid_actionsifa[
'action'] == 'call' )
call_amount = call_action['amount']
If we can call, call instead of folding if call_amountisnotNone : return"call", call_amount
Otherwise keep fold return action, amount
```

Why a Deterministic Bot is Exploitable

The exploit. If our bot always folds hands outside its range, a smart opponent:

1. Observes that the bot folds $\approx 85\%$ of hands from BTN (range $\approx 15\%$)

$$\mathbb{E}[\text{steal}] = 1.5 \times \Pr[\text{bot folds}] > 0$$

Nash Equilibrium requirement. At equilibrium aka optimal play, no player can improve by deviating. A pure fold-outside-range strategy is exploitable thus, *not* an equilibrium and thus, not optimal.

The fix: Minimum Defence Frequency (MDF). We must call with at least $\mathrm{MDF} = P/(P+B)$ of our range to make the steal unprofitable.

Why a Deterministic Bot is Exploitable

The exploit. If our bot always folds hands outside its range, a smart opponent:

1. Observes that the bot folds $\approx 85\%$ of hands from BTN (range $\approx 15\%$)
2. Raises with *any two cards* from the button

$$\mathbb{E}[\text{steal}] = 1.5 \times \Pr[\text{bot folds}] > 0$$

Nash Equilibrium requirement. At equilibrium aka optimal play, no player can improve by deviating. A pure fold-outside-range strategy is exploitable thus, *not* an equilibrium and thus, not optimal.

The fix: Minimum Defence Frequency (MDF). We must call with at least $\mathrm{MDF} = P/(P+B)$ of our range to make the steal unprofitable.

Why a Deterministic Bot is Exploitable

The exploit. If our bot always folds hands outside its range, a smart opponent:

1. Observes that the bot folds $\approx 85\%$ of hands from BTN (range $\approx 15\%$)
2. Raises with *any two cards* from the button
3. Wins the blinds (1.5 bb) every time the bot folds

$$\mathbb{E}[\text{steal}] = 1.5 \times \Pr[\text{bot folds}] > 0$$

Nash Equilibrium requirement. At equilibrium aka optimal play, no player can improve by deviating. A pure fold-outside-range strategy is exploitable thus, *not* an equilibrium and thus, not optimal.

The fix: Minimum Defence Frequency (MDF). We must call with at least $\mathrm{MDF} = P/(P+B)$ of our range to make the steal unprofitable.

MDF: Full Derivation

Setup. Villain bets size B into a pot of P. Let $f = \Pr[\text{we fold}]$ and assume villain's bluff has 0% equity at showdown.

$$\mathbb{E}[\text{bluff}] = B \cdot f - B \cdot (1 - f) \cdot 1 = B(2f - 1)$$

Wait — villain *wins* P if we fold, *loses* B if we call. Correctly:

$$\mathbb{E}[\text{bluff}] = P \cdot f - B \cdot (1 - f)$$

Set $\mathbb{E}[\text{bluff}] = 0$ and solve for the break-even fold frequency:

$$Pf = B(1 - f) \implies f^* = \frac{B}{P + B}$$

So the **minimum call frequency** (MDF) is:

$$\boxed{\mathrm{MDF} = 1 - f^* = \frac{P}{P + B}}$$

If we fold more than f^* of our range, every bluff is instantly profitable regardless of cards.

MDF: Example and Graph

MDF: Example and Graph

Example. Villain bets \$50 into \$100 pot.

$$\mathrm{MDF} = \frac{100}{100 + 50} = \frac{100}{150} = 66.7\%$$

We must call with at least 66.7% of our range.

Interpretation. As bet size B grows relative to pot P, the villain is risking more per bluff, so we can fold *more* and it is still unprofitable. At a pot-sized bet ($B = P$, $\beta = 1$): MDF $= 50\%$.

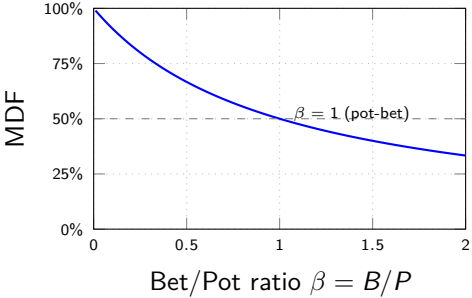
MDF: Example and Graph

Example. Villain bets \$50 into \$100 pot.

$$\mathrm{MDF} = \frac{100}{100+50} = \frac{100}{150} = 66.7\%$$

We must call with at least 66.7% of our range.

Interpretation. As bet size B grows relative to pot P, the villain is risking more per bluff, so we can fold *more* and it is still unprofitable. At a pot-sized bet ($B = P$, $\beta = 1$): MDF $= 50\%$.



Sadly...

While the BluffBot beats the LookupBot more often than not, it still loses with significant probability.

We would like to: (1) understand *exactly* why, and (2) fix it.

Problems with the BluffBot:

- **Uniform randomness is suboptimal.** We should call/bluff with our best trash hands, not uniformly random ones. For example: a *backdoor draw* (needing both turn *and* river to complete a flush or straight, e.g. two hearts in hand with one heart on flop) has ≈3–5% equity contribution — better than pure air. We should defend with these hands first.

Sadly...

While the BluffBot beats the LookupBot more often than not, it still loses with significant probability.

We would like to: (1) understand *exactly* why, and (2) fix it.

Problems with the BluffBot:

- **Uniform randomness is suboptimal.** We should call/bluff with our best trash hands, not uniformly random ones. For example: a *backdoor draw* (needing both turn *and* river to complete a flush or straight, e.g. two hearts in hand with one heart on flop) has ≈3–5% equity contribution — better than pure air. We should defend with these hands first.
- **Equity is ignored.** Calling with K♠2♢to "defend" wastes chips even if it prevents a steal; the EV of the call is negative.

Sadly...

While the BluffBot beats the LookupBot more often than not, it still loses with significant probability.

We would like to: (1) understand *exactly* why, and (2) fix it.

Problems with the BluffBot:

- **Uniform randomness is suboptimal.** We should call/bluff with our best trash hands, not uniformly random ones. For example: a *backdoor draw* (needing both turn *and* river to complete a flush or straight, e.g. two hearts in hand with one heart on flop) has ≈3–5% equity contribution — better than pure air. We should defend with these hands first.
- **Equity is ignored.** Calling with K♠2♢to "defend" wastes chips even if it prevents a steal; the EV of the call is negative.
- **15% is arbitrary.** We have not solved for the GTO call frequency.

Sadly...

While the BluffBot beats the LookupBot more often than not, it still loses with significant probability.

We would like to: (1) understand *exactly* why, and (2) fix it.

Problems with the BluffBot:

- **Uniform randomness is suboptimal.** We should call/bluff with our best trash hands, not uniformly random ones. For example: a *backdoor draw* (needing both turn *and* river to complete a flush or straight, e.g. two hearts in hand with one heart on flop) has ≈3–5% equity contribution — better than pure air. We should defend with these hands first.
- **Equity is ignored.** Calling with K♠2♢to "defend" wastes chips even if it prevents a steal; the EV of the call is negative.
- **15% is arbitrary.** We have not solved for the GTO call frequency.
- **Bet sizing is ignored.** A min-raise deserves a different response than a 3x raise.

Table of Contents

1. Hand Strength
2. Tight vs. Loose Play & Range Analysis
3. Positional Advantage
4. Building the Lookup Bot
5. Bluffing & the 15% Surprise
6. Next Time: Bluffing Theory & the Road to GTO

Next Week

We will come back to the BluffBot and make it rigorous.

1. **Bluffing theory**: which hands to bluff/call with, not just how often

The 15% Bot beats the Lookup Bot. Both lose to a GTO solver.
Why? What exactly is GTO? We will find out.

Next Week

We will come back to the BluffBot and make it rigorous.

1. **Bluffing theory**: which hands to bluff/call with, not just how often
2. **Bet sizing and polarisation**: why sizing carries range information

The 15% Bot beats the Lookup Bot. Both lose to a GTO solver.
Why? What exactly is GTO? We will find out.

Next Week

We will come back to the BluffBot and make it rigorous.

1. **Bluffing theory**: which hands to bluff/call with, not just how often
2. **Bet sizing and polarisation**: why sizing carries range information
3. **Alpha (fold equity)**: when is a bluff profitable, precisely

The 15% Bot beats the Lookup Bot. Both lose to a GTO solver.
Why? What exactly is GTO? We will find out.

Next Week

We will come back to the BluffBot and make it rigorous.

1. **Bluffing theory**: which hands to bluff/call with, not just how often
2. **Bet sizing and polarisation**: why sizing carries range information
3. **Alpha (fold equity)**: when is a bluff profitable, precisely
4. **Moving toward GTO**: Nash equilibrium strategies

The 15% Bot beats the Lookup Bot. Both lose to a GTO solver.
Why? What exactly is GTO? We will find out.

Next Week

We will come back to the BluffBot and make it rigorous.

1. **Bluffing theory**: which hands to bluff/call with, not just how often
2. **Bet sizing and polarisation**: why sizing carries range information
3. **Alpha (fold equity)**: when is a bluff profitable, precisely
4. **Moving toward GTO**: Nash equilibrium strategies
5. **CFR intro**: how computers actually solve poker

The 15% Bot beats the Lookup Bot. Both lose to a GTO solver.
Why? What exactly is GTO? We will find out.

Questions?

Next Week: Bluffing, Bet Sizing & the Road to GTO.